



Especialización en Diseño y Desarrollo de Videojuegos
Facultad de ingeniería

ACA 1: Diseñar la arquitectura orientada a objetos de un videojuego tipo Breakout en Lua, modelando entidades, responsabilidades y máquinas de estado en formato UML y pseudocódigo

Presentado por:

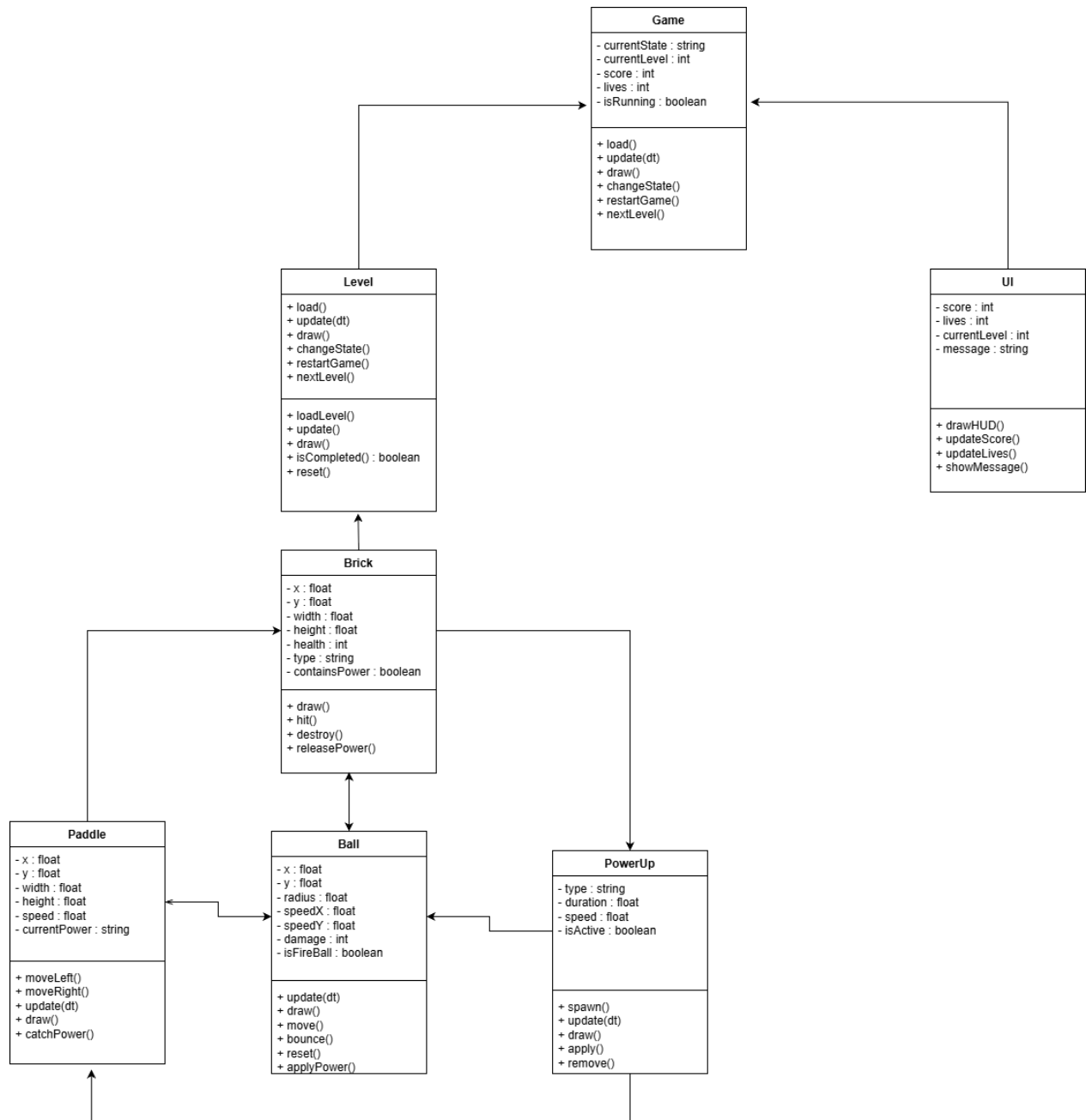
Jhoan Andrés Castelblanco Utria

Julian Eleicer Orellanos

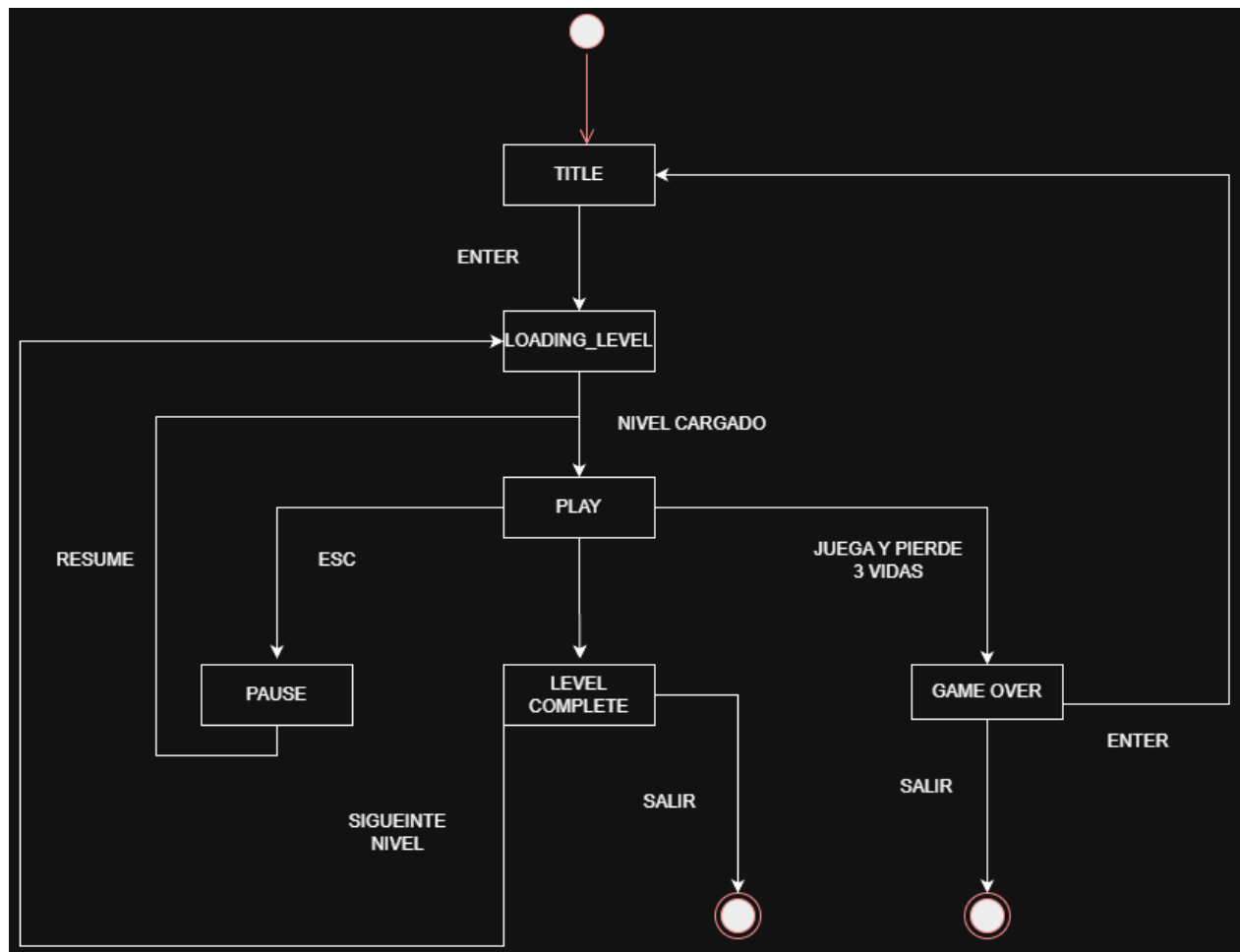
Bogotá, Cundinamarca 26 de julio de 2023

Repositorio Github: <https://github.com/johancastel/Breakout-Guardians-of-the-Forge>

1. Diagrama UML de clases del Breakout.



2. Diagrama de la máquina de estados del juego.



3. Tabla de responsabilidades por clase (clase / atributos / métodos / colaboradores).

table of responsible part - Breakout: Guardians of the Forge				
Clase	Responsabilidad	Atributos	Métodos	Colaboradores
Game	Administrar el flujo general del videojuego, controlar los estados, iniciar partidas y coordinar las entidades principales.	currentState, score, lives	load(), update(), draw(), restartGame()	Level, UI, Ball
Ball	Representar la bola principal (Energy Core), controlar su movimiento, detectar colisiones y aplicar efectos especiales.	x, y, radius, speedX, speedY	move(), bounce(), draw()	Paddle, Brick, PowerUp
Paddle	Representar el Forge Shield controlado por el jugador y permitir el desplazamiento horizontal para dirigir la bola.	x, y, width, speed	moveLeft(), moveRight(), catchPower()	Ball, PowerUp
Brick	Representar los bloques del nivel, administrar su resistencia y liberar poderes al ser destruidos.	health, type, containsPower	hit(), destroy(), releasePower()	Ball
Level	Gestionar la información de cada nivel, cargar los bloques y verificar cuándo un nivel ha sido completado.	levelNumber, bricks, difficulty	loadLevel(), update(), isCompleted()	Brick, Game
UI	Mostrar la interfaz gráfica del juego, incluyendo puntaje, vidas, nivel actual y mensajes al jugador.	score, lives, currentLevel	drawHUD(), updateScore()	Game
PowerUp	Administrar los poderes especiales que modifican temporalmente el comportamiento de la bola o la barra.	type, duration, isActive	apply(), remove(), update()	Ball, Paddle, Brick

4. Pseudocódigo del Game Loop principal.

Algoritmo General

INICIO

Inicializar el juego

Cargar recursos del juego

- Sprites
- Sonidos
- Fuentes
- Configuración

Establecer el estado inicial como TITLE

Mientras la aplicación esté en ejecución hacer

Leer la entrada del jugador

Según el estado actual del juego hacer

TITLE

Mostrar pantalla principal

Si el jugador presiona ENTER entonces

Cambiar estado a LOADING_LEVEL

Fin Si

LOADING_LEVEL

Cargar el nivel actual

Crear los bloques

Inicializar la pelota

Posicionar la barra

Inicializar la interfaz

Cambiar estado a PLAY

PLAY

Actualizar la barra

Actualizar la pelota

Detectar colisiones

Actualizar bloques

Actualizar poderes

Actualizar la interfaz

Dibujar todos los elementos del juego

Si el jugador presiona ESC entonces

 Cambiar estado a PAUSE

Fin Si

Si el jugador pierde todas las vidas entonces

 Cambiar estado a GAME_OVER

Fin Si

Si todos los bloques fueron destruidos entonces

 Cambiar estado a LEVEL_COMPLETE

Fin Si

PAUSE

 Mostrar menú de pausa

 Esperar la decisión del jugador

 Si el jugador selecciona CONTINUAR entonces

 Cambiar estado a PLAY

 Fin Si

 Si el jugador selecciona SALIR entonces

 Finalizar aplicación

Fin Si

LEVEL_COMPLETE

Mostrar mensaje de nivel completado

Calcular bonificación

Incrementar el número del nivel

Si existen más niveles entonces

Cambiar estado a LOADING_LEVEL

En caso contrario

Cambiar estado a TITLE

Fin Si

GAME_OVER

Mostrar pantalla de Game Over

Mostrar puntaje final

Si el jugador presiona ENTER entonces

Reiniciar partida

Cambiar estado a TITLE

Fin Si

Si el jugador selecciona SALIR entonces

Finalizar aplicación

Fin Si

Fin Según

Fin Mientras

FIN

5. Reflexión

Hacer este ACA real mente si nos parecio muy fascinante ya que no lo tomamos como solo un entregable sino por el contrario como un producto que debe tener inicio y debe tener fin, claramente no se me está solicitando hacer un juego, ¡pero!, si buscamos buenas prácticas de creación, estructura y organización para la creación de un video juego, claro, falta mucho como bases de datos GDD entre muchas cosas, pero quería estructurar un proyecto que tenga una vista seria y responsable.

Ya en materia de aprendizaje comprendimos que el diseño de un videojuego comienza mucho antes de escribir código. Al iniciar el trabajo teníamos varias dudas sobre los entregables solicitados, especialmente porque confundíamos el diagrama UML con el pseudocódigo. A medida que desarrollamos la actividad entendimos que el UML permite representar la estructura del videojuego mediante clases, atributos, métodos y relaciones, mientras que el pseudocódigo describe la lógica del funcionamiento del juego y del Game Loop sin depender de un lenguaje de programación.

Otro aspecto nuevo para nosotros fue la tabla de responsabilidades por clase. Antes de esta actividad no conocíamos este tipo de documento ni su importancia dentro del diseño orientado a objetos. Al elaborarla comprendimos que ayuda a definir claramente la función de cada clase, facilitando la organización del proyecto y la creación del UML y evitando mezclar responsabilidades entre las diferentes entidades del videojuego.

Otro aprendizaje importante fue volver a utilizar archivos con formato **Markdown (.md)** para documentar el proyecto. Aunque ya conocíamos este formato, hacía bastante tiempo no lo utilizábamos y esta actividad permitió recordar nuevamente su sintaxis y las ventajas que ofrece para organizar documentación técnica dentro de un proyecto de software. Gracias al uso de Markdown fue posible estructurar el concepto del juego, las entidades y el pseudocódigo de una manera clara, ordenada y fácil de mantener, tal como se realiza en muchos proyectos reales de desarrollo de software.

Adicionalmente, esta actividad permitió tener un primer acercamiento a **Lua** y al motor **Love2D**. Aunque todavía no desarrollamos el videojuego como tal, fue interesante comprender la estructura básica que debe tener un proyecto, conocer la función de archivos como `main.lua` y `conf.lua`, así como entender que Love2D utiliza funciones específicas como `love.load()`, `love.update()` y `love.draw()` para controlar el ciclo principal del juego. Este primer contacto con la sintaxis de Lua despertó aún más nuestro interés por continuar aprendiendo el lenguaje durante el desarrollo de la especialización.